

Design Document using SA/SD Methodology

for

Secure File Sharing with Encryption

Chandan Gowda C (241IT016), Prabhav P (241IT053),
Sucheth K Katte (241IT078)

August 17, 2026

Contents

1	Data Flow Diagram (DFD) Model	2
1.1	Level 0 DFD	2
1.1.1	Context Diagram	2
1.2	Level 1 DFD	3
1.2.1	Top-Level Functional Decomposition	3
1.3	Level 2 DFDs	4
1.3.1	0.1 User Registration & Authentication	4
1.3.2	0.2 Public Key Directory Lookup	5
1.3.3	0.3 File Upload & Encryption (Sender Side)	6
1.3.4	0.4 Delivery Management	7
1.3.5	0.5 File Decryption & Download (Receiver Side)	8
1.3.6	0.6 Metadata Logging & Audit	9
1.3.7	0.7 Error Handling	10
2	Core Modules	11
2.1	0.1 User Registration & Authentication	11
2.2	0.2 Public Key Directory Lookup	11
2.3	0.3 File Upload & Encryption (Sender Side)	11
2.4	0.4 Delivery Management	11
2.5	0.5 File Decryption & Download (Receiver Side)	11
2.6	0.6 Metadata Logging & Audit	11
2.7	0.7 Error Handling	12
3	Data Dictionary	13
3.1	Data Stores	13
3.2	Data Flows	13
3.3	Primitive Data Item Types	14
4	Structured Design: Structure Chart	15

1 Data Flow Diagram (DFD) Model

1.1 Level 0 DFD

1.1.1 Context Diagram

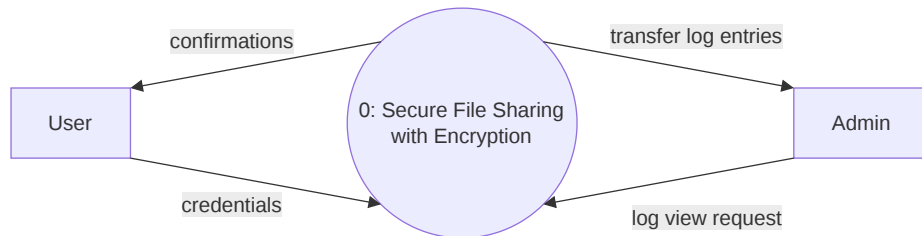


Figure 1: Level 0 Context Diagram – the entire system as a single process, showing only external entities

1.2 Level 1 DFD

1.2.1 Top-Level Functional Decomposition

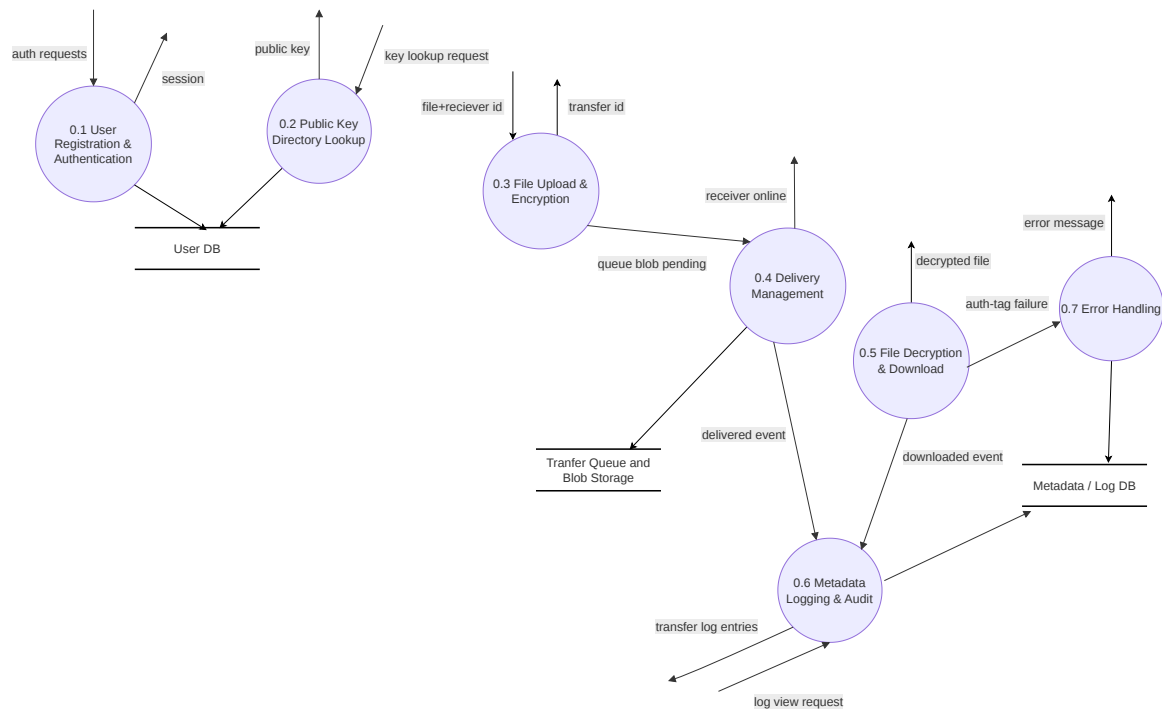


Figure 2: Level 1 DFD – process 0 decomposed into 7 functional groups

1.3 Level 2 DFDs

1.3.1 0.1 User Registration & Authentication

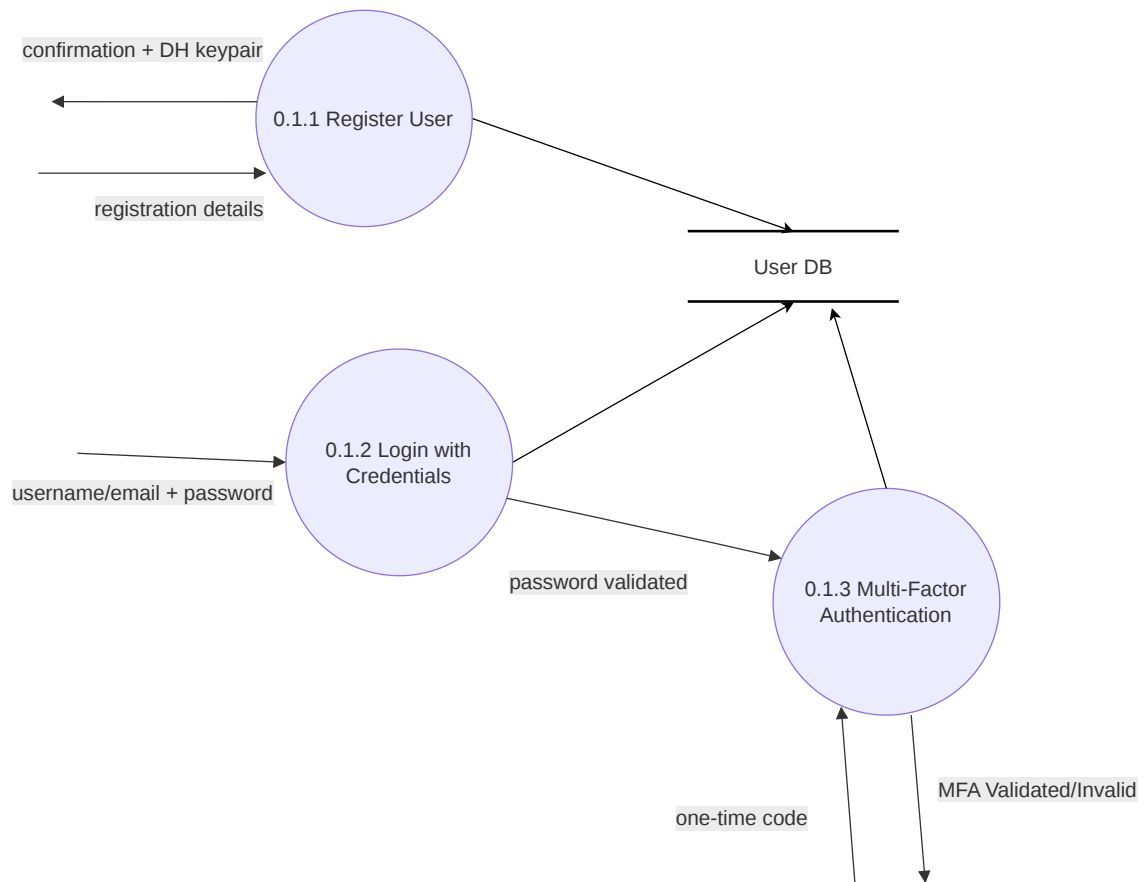


Figure 3: Level 2 DFD for 0.1 User Registration & Authentication

1.3.2 0.2 Public Key Directory Lookup

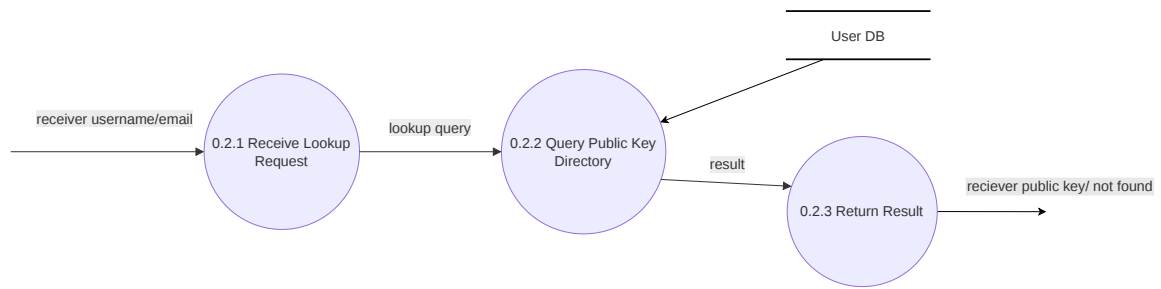


Figure 4: Level 2 DFD for 0.2 Public Key Directory Lookup

1.3.3 0.3 File Upload & Encryption (Sender Side)

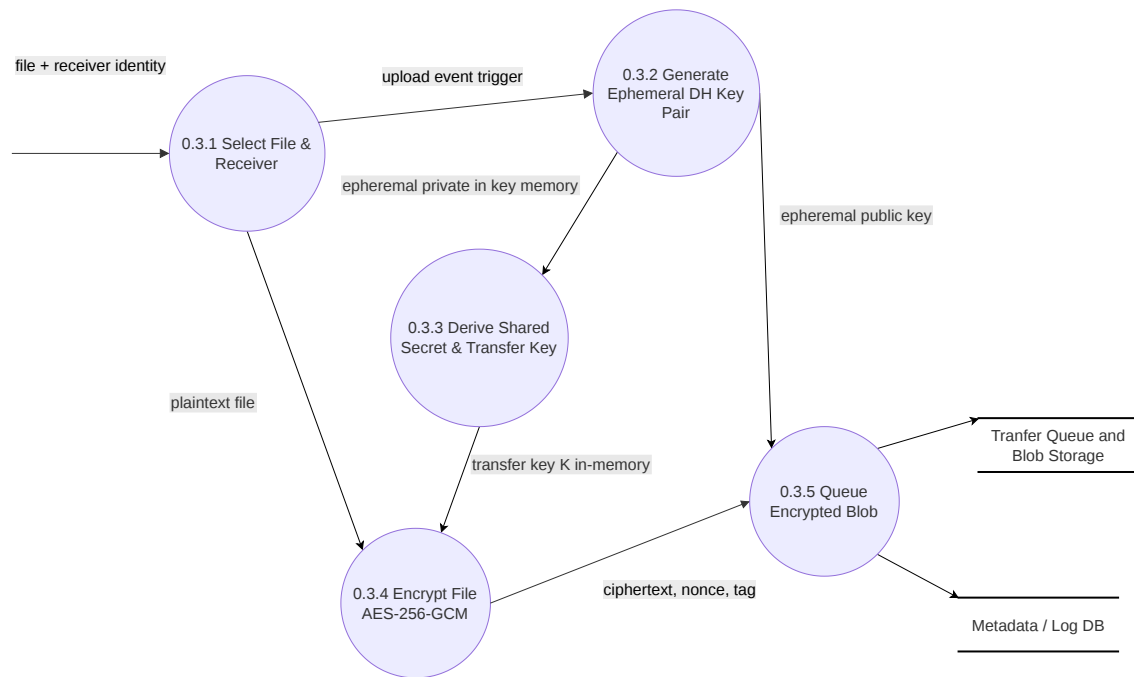


Figure 5: Level 2 DFD for 0.3 File Upload & Encryption (Sender Side)

1.3.4 0.4 Delivery Management

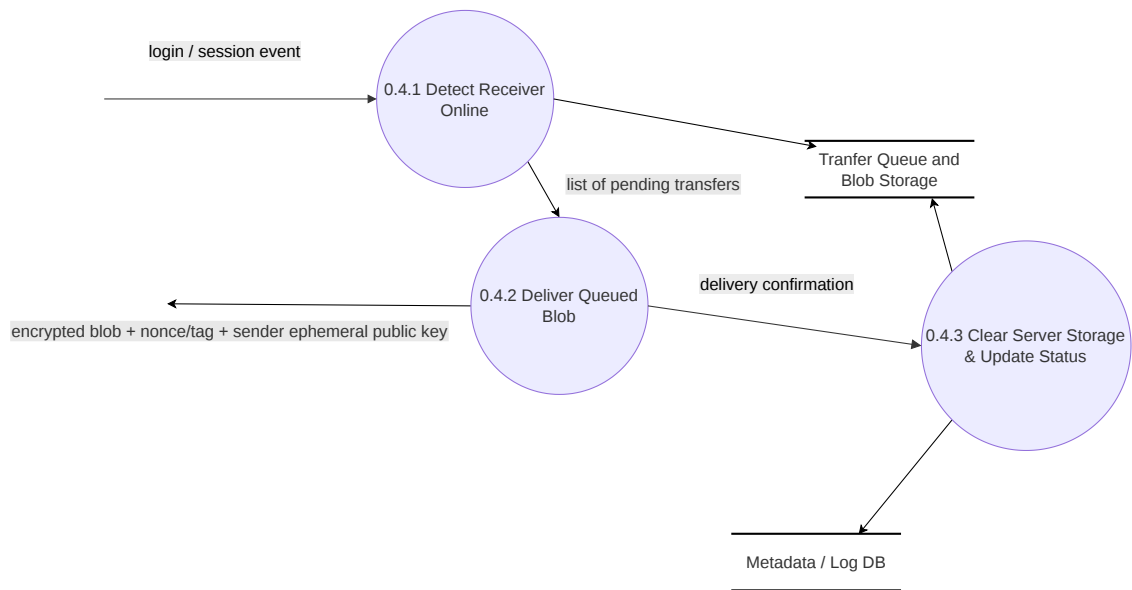


Figure 6: Level 2 DFD for 0.4 Delivery Management

1.3.5 0.5 File Decryption & Download (Receiver Side)

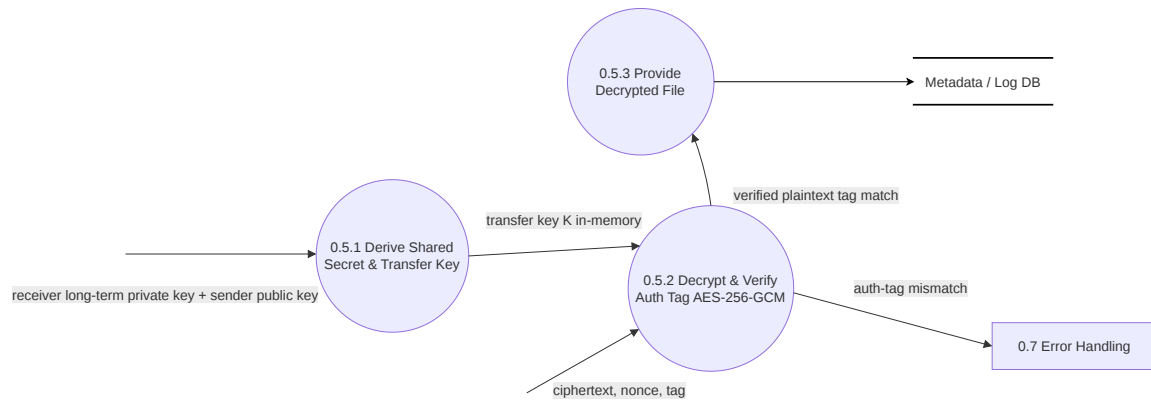


Figure 7: Level 2 DFD for 0.5 File Decryption & Download (Receiver Side)

1.3.6 0.6 Metadata Logging & Audit

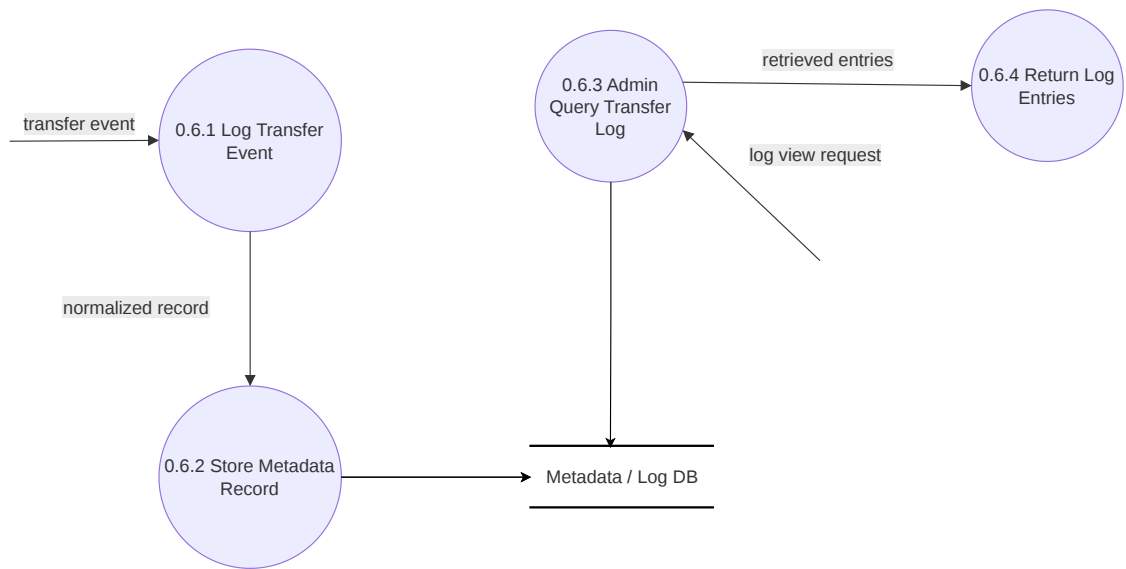


Figure 8: Level 2 DFD for 0.6 Metadata Logging & Audit

1.3.7 0.7 Error Handling

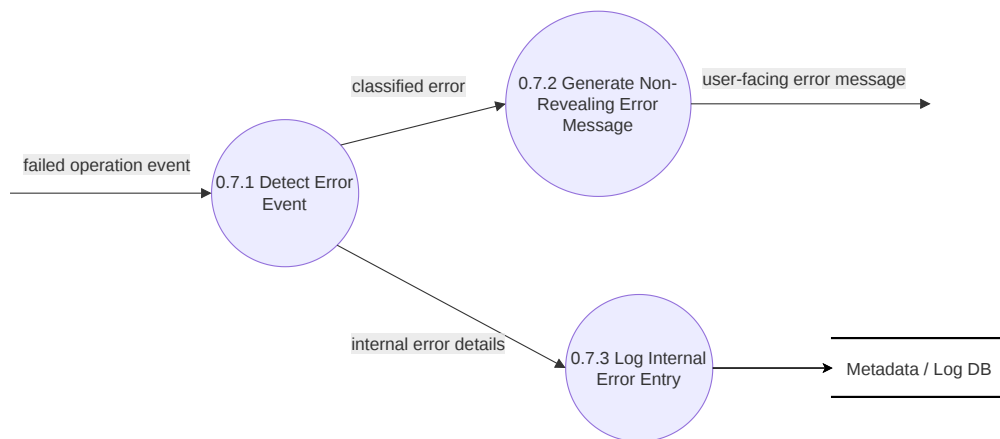


Figure 9: Level 2 DFD for 0.7 Error Handling

2 Core Modules

This section describes the main modules of the Secure File Sharing with Encryption system. Each module has been assigned to a designer for detailed implementation, and the functional requirements (from the SRS) realised by each module are listed.

2.1 0.1 User Registration & Authentication

Handles new user registration (with client-side long-term DH key-pair generation), credential-based login, and mandatory multi-factor authentication.

Designer: Prabhav P

Functions: Register User, Login with Email/Password, Multi-Factor Authentication

Requirements Covered: F.1, F.2, F.3

2.2 0.2 Public Key Directory Lookup

Allows an authenticated sender to look up the long-term public key of an intended receiver from the server-side key directory.

Designer: Prabhav P

Functions: Receive Lookup Request, Query Public Key Directory, Return Result

Requirements Covered: F.4

2.3 0.3 File Upload & Encryption (Sender Side)

Generates a fresh ephemeral DH key pair per transfer, derives the AES-256 transfer key via Diffie-Hellman and HKDF, encrypts the file with AES-256-GCM, and places the encrypted blob into the delivery queue.

Designer: Prabhav P

Functions: Select File & Receiver, Generate Ephemeral DH Key Pair, Derive Shared Secret & Transfer Key, Encrypt File with AES-256-GCM, Queue Encrypted File for Delivery

Requirements Covered: F.5, F.6, F.7, F.8, F.9

2.4 0.4 Delivery Management

Detects when the intended receiver comes online, delivers the queued encrypted blob to them over HTTPS, and clears the server-side copy once delivery is confirmed.

Designer: Chandan Gowda C

Functions: Detect Receiver Online & Trigger Delivery, Deliver Queued File & Clear Server Storage

Requirements Covered: F.10, F.11

2.5 0.5 File Decryption & Download (Receiver Side)

Re-derives the shared secret and transfer key on the receiver's device, decrypts the file using AES-256-GCM while verifying the authentication tag, and provides the plaintext file for download.

Designer: Chandan Gowda C

Functions: Derive Shared Secret & Transfer Key (Receiver Side), Decrypt and Download File

Requirements Covered: F.12, F.13

2.6 0.6 Metadata Logging & Audit

Logs every upload, queue, delivery and download event as a metadata record (without storing plaintext content, private keys or shared secrets), and allows administrators to view the transfer

log.

Designer: Sucheth K Katte

Functions: Metadata Logging, Admin Transfer Log Viewing

Requirements Covered: F.14, F.15

2.7 0.7 Error Handling

Detects and reports errors such as failed authentication-tag verification, corrupted uploads, unregistered receiver keys, or storage/queue failures, using clear, non-revealing error messages that do not leak cryptographic material.

Designer: Sucheth K Katte

Functions: Error Handling

Requirements Covered: F.16

3 Data Dictionary

A single, consolidated data dictionary is given below, covering every data flow and data store appearing across all DFDs in Section 1 (Level 0 through Level 2). Composition operators used: + (composition), [,] (selection), () (optional data), { } (iteration), = (equivalence), and /* */ (comment).

3.1 Data Stores

Data Store	Definition
User DB	User-DB = {username + email + hashed-password + MFA-status + long-term-public-key}* *
Transfer Queue and Blob Storage	Transfer-Queue = {ciphertext + nonce + tag + sender-ephemeral-public-key + receiver-id + status}* *
Metadata / Log DB	Metadata-Log = {transfer-id + sender + receiver + timestamp + status}* *

3.2 Data Flows

Data Item	Definition / Composition
credentials	credentials = [username, email] + account-password
registration-details	registration-details = username + email + account-password
confirmation + DH-keypair	= registration-confirmation-msg + long-term-public-key /* private key retained client-side only */
one-time-code	one-time-code: integer /* MFA verification code */
MFA-status	MFA-status = [validated, invalid]
auth-requests	auth-requests = credentials + (one-time-code)
session	session = session-token + expiry
public-key	public-key: {byte}* /* DH public key, long-term or ephemeral */
private-key	private-key: {byte}* /* never transmitted or stored on server */
key-lookup-request	key-lookup-request = [receiver-username, receiver-email]
result	result = [receiver-public-key, not-found-msg]
file+receiver-id	file+receiver-id = plaintext-file + receiver-identity
plaintext-file	plaintext-file: {byte}*
ephemeral-key-pair	ephemeral-key-pair = ephemeral-private-key + ephemeral-public-key /* generated fresh per transfer, F.6 */
shared-secret	shared-secret: {byte}* /* computed in-memory only, never persisted */
transfer-key-K	transfer-key-K: {byte}256 /* AES-256 key derived via HKDF(shared-secret) */

Data Item	Definition / Composition
ciphertext+nonce+tag	= file-ciphertext + nonce + auth-tag
transfer-id	transfer-id: integer /* unique per transfer, for audit/tracking */
receiver-online-event	receiver-online-event = receiver-id + login-timestamp
pending-transfers-list	pending-transfers-list = {transfer-id}*
delivered-event	delivered-event = transfer-id + delivery-timestamp + delivery-status
decrypted-file	decrypted-file = plaintext-file
downloaded-event	downloaded-event = transfer-id + download-timestamp
auth-tag-mismatch	auth-tag-mismatch = transfer-id + error-code
error-message	error-message = [auth-tag-mismatch, corrupted-upload-msg, receiver-not-found-msg, storage-failure-msg]
transfer-event	transfer-event = [upload-event, queued-event, delivered-event, downloaded-event]
normalized-record	normalized-record = transfer-id + sender + receiver + timestamp + status
log-view-request	log-view-request = (date-range) + (username-filter)
transfer-log-entries	transfer-log-entries = {normalized-record}*

3.3 Primitive Data Item Types

Primitive	Type
username, email	string
account-password	string /* stored server-side only as a salted hash, never in plaintext, and never used to derive any file-encryption key – C.7 */
timestamp	datetime
status	[pending, delivered, downloaded]

4 Structured Design: Structure Chart

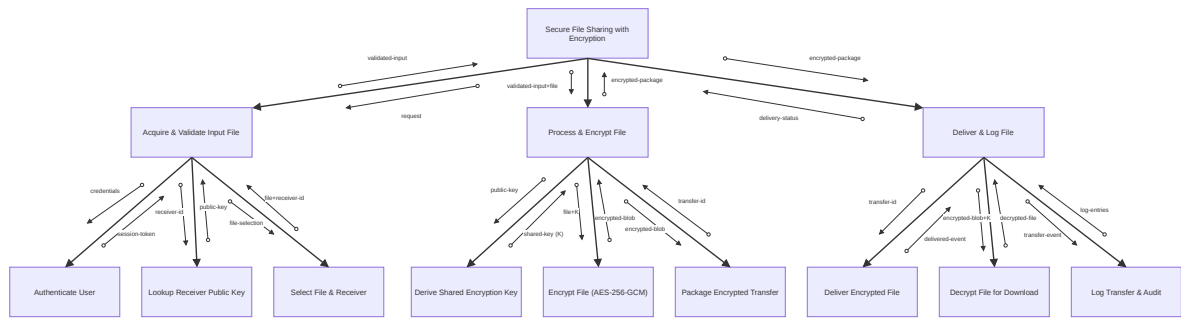


Figure 10: Structure Chart of the System